Program 2: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Step 1: To create a Maven project run the following command in terminal:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp - DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

This command generates a basic Maven project structure with a sample App.java and AppTest.java.

```
vboxuser@prathwish03:~$ mvn archetype:generate \
  -DgroupId=com.example \
  -DartifactId=MyMavenApp \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/vboxuser
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vboxuser/MyMavenApp
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
```

This “BUILD SUCCESS” will confirm that the project is created successfully

```
vboxuser@prathwish03:~/devops$ cd MyMavenApp
vboxuser@prathwish03:~/devops/MyMavenApp$ ls
pom.xml  src  target
vboxuser@prathwish03:~/devops/MyMavenApp$
```

Step 2: Go to the project directory using command **cd MyMavenApp**

Step 3: Open and edit pom.xml file using command **gedit pom.xml**

- Go to https://mvnrepository.com
- Search for a dependency “junit”
- Select the latest version and copy the dependency XML snippet.
- Paste it under the <dependencies> section in pom.xml
- Updated pom.xml:

```
<project
xmlns="http://maven.apache.org/POM/
4.0.0"
xmlns:xsi="http://www.w3.org/2001/X
MLSchema-instance"
xsi:schemaLocation="http://maven.apac
he.org/POM/4.0.0
http://maven.apache.org/maven-
v4_0_0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>MyMavenApp</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>
<name>MyMavenApp</name><url>http://maven.apache.org</url>
<!-- Java version configuration -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
<!-- Dependencies -->
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
</dependencies>
<!-- Build Plugins -->
<build>
<plugins>
<!-- Compiler Plugin -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler- plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>11</source>
```

```
<target>11</target>
</configuration>
</plugin>
<!-- Surefire Plugin for Testing -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<!-- JAR Plugin with Entry Point -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<version>3.1.0</version>
<configuration>
<archive>
<manifestEntries>
<Main-Class>com.example.App</Main-Class>
</manifestEntries>
</archive></configuration>
</plugin>
</plugins>
</build>
</project>
```

Step 4: Build the project using command **mvn clean install**

The "Build Success" message ensures that our Maven project is built successfully.

Step 5: To run the Maven application type he command:

java -jar target/MyMavenApp-1.0-SNAPSHOT.jar

- After completing the execution, the output will look like:

```
vboxuser@prathwish03:~/devops/MyMavenApp$ java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
Hello World!
vboxuser@prathwish03:~/devops/MyMavenApp$
```

This will show output as the code present in App.java file we can edit the text to get any other output.

Creating a Maven Project with Guava and Commons-IO

Google Guava Dependency

Guava is a set of core libraries developed by Google to fill gaps in

Java's standard libraries. It is used for:

- Collections enhancements: Multimap, BiMap, Table, ImmutableList, etc.
- Functional programming utilities: Predicates, Functions, Suppliers.
- Caching: Built-in caching mechanisms (CacheBuilder) like a mini in-memory cache.String utilities: Splitter, Joiner, CharMatcher.
- Concurrency: ListenableFuture, Service, and other helper classes.
- Hashing and hashing strategies: Useful for consistent

hashing (e.g., Bloom filters). Overall, Guava simplifies and

improves many core Java patterns and utilities.

Apache Commons IO Dependency

Apache Commons IO is focused purely on input/output utilities, especially

for file and stream handling. It is used for:

- File operations: Easily read/write files with FileUtils.
- Stream utilities: IOUtils.copy(), IOUtils.toString() — simplifies stream handling.
- Directory monitoring: FileAlterationObserver for detecting changes in a folder.
- File filtering: WildcardFileFilter, SuffixFileFilter.

Overall, Commons IO reduces boilerplate and errors when working with files or streams.

Maven Project with Guava and Commons-IO Dependencies

Create a Maven Project Using Terminal by following command:

mvn archetype: generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp - DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
vboxuser@prathwish03:~/devops$ mvn archetype:generate   -DgroupId=com.example   -DartifactId=MyMavenGuavaApp   -Darc
etypeArtifactId=maven-archetype-quickstart   -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/vboxuser/devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenGuavaApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vboxuser/devops/MyMavenGuavaApp
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
[INFO] Total time:  4.354 s
[INFO] Finished at: 2026-03-21T09:56:06+05:30
```

Now Navigate to the Project Directory using **cd MyMavenGuavaApp**

Edit the pom.xml File and add the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven- v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId><artifactId>MyMavenGuavaApp</artifactId
    >
    <packaging>jar</packaging>
    <version>1.0-SNAPSHOT</version>
    <name>MyMavenGuavaApp</name>
    <url>http://maven.apache.org</url>
    <dependencies>
      <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>3.8.1</version>
        <scope>test</scope>
      </dependency>

      <!-- https://mvnrepository.com/artifact/com.google.guava/guava -->
    <dependency>
```

```
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.4.0-jre</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.18.0</version>
</dependency>

  </dependencies>

  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
           <archive>
              <manifestEntries>
                 <Main-Class>com.example.App</Main-Class>
```

```
            </manifestEntries>
          </archive>
        </configuration>
      </plugin>
  </plugins>
  </build>
  </project>
```

Modify App.java

Got to App.java using **cd src/main/java/com/example/** and **gedit App.java** to type the file with following code:

```
package com.example;

/**
 * Hello world!
 *
 */
// Import FileUtils class from Apache Commons IO library
// This is used to perform file operations
like copying files import
org.apache.commons.io.FileUtils;

// Import File class to represent file and
directory pathnames import
java.io.File;

// Import IOException to handle
input-output errors import
java.io.IOException;

// Import ImmutableList from Google Guava library
// ImmutableList creates a list that cannot be modified after creation
```

```
//import com.google.common.collect.ImmutableList;

// Define the
main class
public class
App
{
   // Main method - program execution
   starts from here public static void
   main( String[] args )
   {
      // Create an immutable list of strings (fruits)
      // This list cannot be changed (no add/remove allowed)
      ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana",
      "Cherry");

      // Print the list of fruits to the
      console System.out.println(fruits);

      // Create a File object
      representing the source file File
      sourceFile = new
      File("source.txt");

      // Create a File object representing
      the destination file File destFile =
      new File("destination.txt");

      try {
         // Copy the contents of source.txt to destination.txt
         // Using Apache Commons IO FileUtils
         class FileUtils.copyFile(sourceFile,
```

```
            destFile);

            // Print success message if file is copied
            System.out.println("File copied
            successfully!");
        }
        catch (IOException e) {
            // If any error occurs during file copying,
            // this block will handle the exception
            // Print error message
            System.err.println("Error occurred while copying file: " +
            e.getMessage());
        }
    }
}
```

Build the Project using the command **mvn clean install** command

```
vboxuser@prathwish03:~/devops/MyMavenGuavaApp$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:MyMavenGuavaApp >--------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenGuavaApp ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/devops/MyMavenGuavaApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenGuavaApp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/vboxuser/devops/MyMavenGuavaApp/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/devops/MyMavenGuavaApp/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenGuavaApp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/vboxuser/devops/MyMavenGuavaApp/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenGuavaApp ---
[INFO]
[INFO] -------------------------------------------------------
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.059 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenGuavaApp ---
[INFO] Building jar: /home/vboxuser/devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenGuavaApp ---
[INFO] Installing /home/vboxuser/devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar to /home/vboxuser/.m2/re
App/1.0-SNAPSHOT/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/vboxuser/devops/MyMavenGuavaApp/pom.xml to /home/vboxuser/.m2/repository/com/example/MyMavenGuav
-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  10.495 s
[INFO] Finished at: 2026-03-21T10:02:34+05:30
[INFO] ------------------------------------------------------------------------
```

This compiles the project and creates a JAR in the target/ folder.

Run the Application:

If java-jar command give an error which is most probable, use the following command

mvn exec:java -Dexec.mainClass=”com.example.App”

```
vboxuser@prathwish03:~/devops/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass=com.example.App
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:MyMavenGuavaApp >--------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenGuavaApp ---
[Apple, Banana, Cherry]
Error occurred while copying file: Source 'source.txt' does not exist
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.704 s
[INFO] Finished at: 2026-03-21T11:25:10+05:30
[INFO] ------------------------------------------------------------------------
```

This command gives error asError while copying file:Source ‘source.txt’ does not exist

Now, Create a Sample File to copy the content to destination file

```
Open    source.txt
        ~/report/MyMavenGuavaApp    Save
1 Copying contents of this file to the destination file
```

Now Run the command **mvn exec:java -Dexec.mainClass=”com.example.App”** again

```
vboxuser@prathwish03:~/devops/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass=com.example.App
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:MyMavenGuavaApp >--------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenGuavaApp ---
[Apple, Banana, Cherry]
File copied successfully!
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.623 s
[INFO] Finished at: 2026-03-21T11:28:19+05:30
[INFO] ------------------------------------------------------------------------
```

The output shows file copied successfully.

We can see newly created **destination.txt** using gedit destination.txt

if you are still in Project’s directory

```
vboxuser@prathwish03:~/devops/MyMavenGuavaApp$ ls
destination.txt  pom.xml  source.txt  src  target
vboxuser@prathwish03:~/devops/MyMavenGuavaApp$
```

File content is same as the content in source file.

Pipeline For MyMavenApp

Step 1: Create new file named "**Jenkinsfile**" inside MyMavenApp folder and push into git

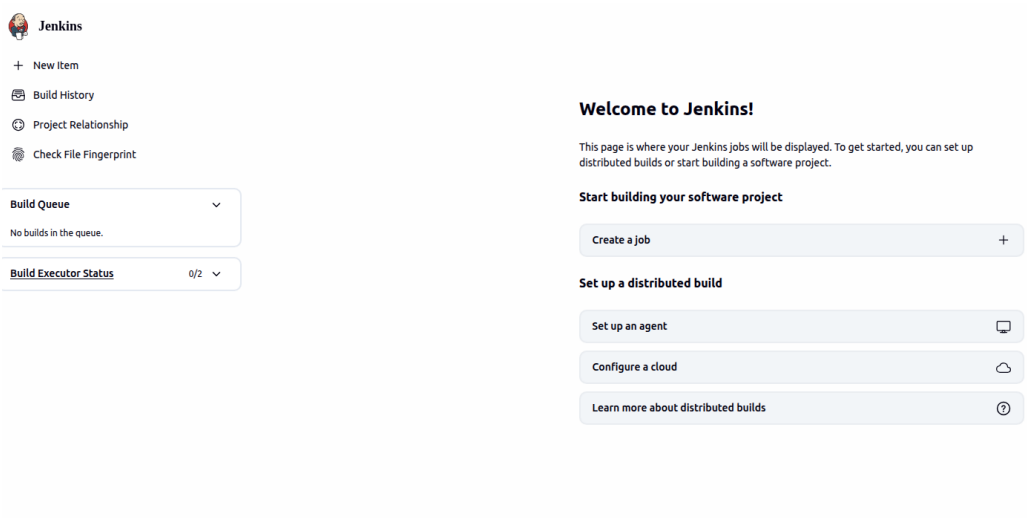
Jenkinsfile:

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/PrathwishShetty03/MavenApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }
        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
            }
        }
    }

    post {
        success {
            echo 'Build and deployment successful!'
```

```
        }
        failure {
            echo 'Build failed!'
        }
    }
}
```

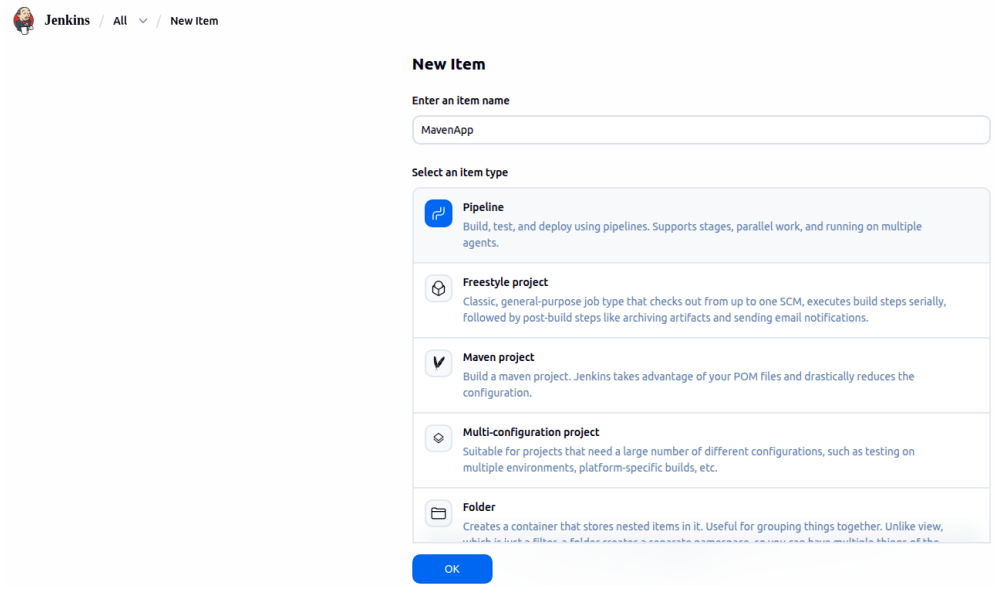
Step 2:Open Jenkins Dashboard

- Open Jenkins in browser
- Click on New Item



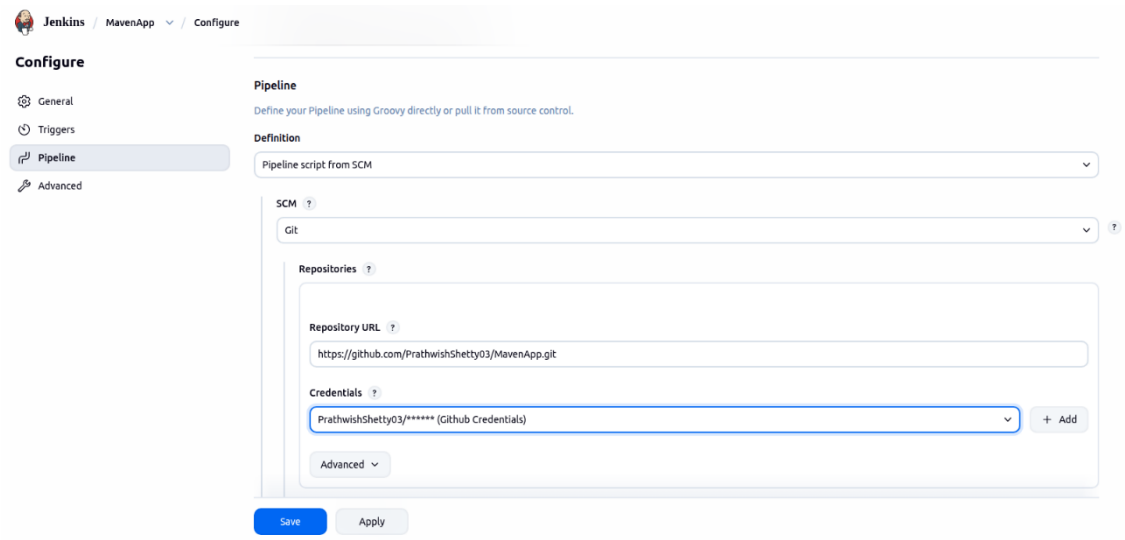
Step 3: Create Pipeline Job

- Enter name: **MavenApp**
- Select **Pipeline**
- Click **OK**



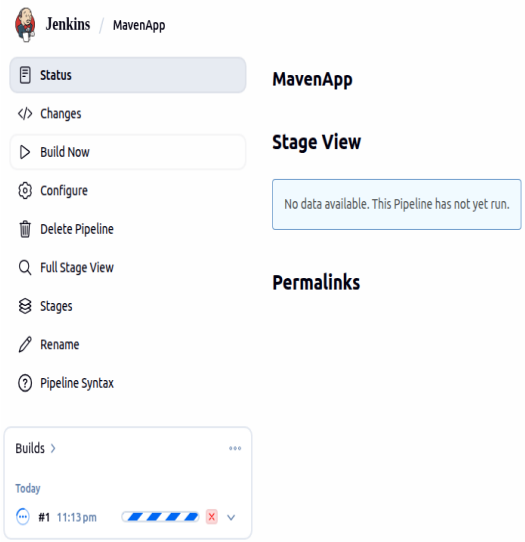
Step 4: Configure Pipeline with GitHub

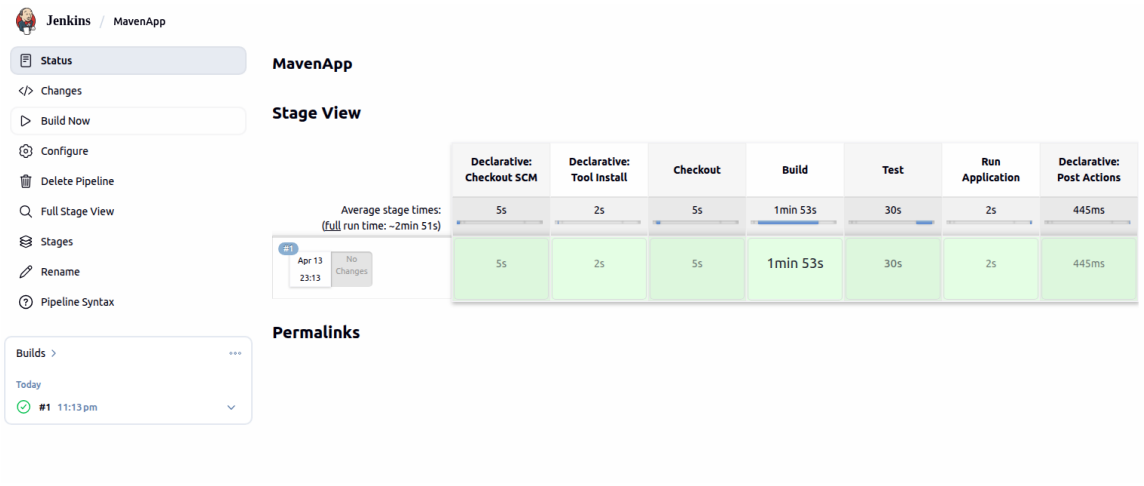
- Select **Pipeline script from SCM**
- SCM → Git
- Enter repository URL
- Add credentials
- Click **Save**



Step 5: Run the Pipeline

- Click **Build Now**



Step 6: Look for the status of project

- Check Stage View
- View Build History